

IBVAP — SHA-256 Audit Log & Cryptographic Integrity

08. Cryptographic Hash-Chaining & Tamper Detection

1. How the Cryptographic Audit Chain Works

To prevent internal actors or external hackers from modifying, inserting, or deleting security breach logs, IBVAP implements a **SHA-256 Hash Chain** across all recorded security events.

Hash Chaining Formula

For any record $i \geq 1$:

- Construct normalized JSON string payload R_i from fields: **event_id**, **event_type**, **person_track_id**, **frame**, **timestamp**, **risk**.

$R_i = \text{json.dumps}(\text{record}_i, \text{sort_keys}=\text{True}, \text{separators}=(", ", ":"))$

- Concatenate the previous record's hash H_{i-1} with string R_i :

$\text{Input}_i = H_{i-1} + R_i$

- Compute the record's SHA-256 hash digest H_i :

$H_i = \text{SHA256}(\text{Input}_i)$

For the first record ($i=1$), $H_0 = \text{"GENESIS"}$.

2. Actual Project Audit Chain (Verified from Source)

Source files: **output/secure_audit_log.csv** & **output/audit_root_hash.txt**

Chain Data Table

Record #	Event ID	Person Track ID	Frame	Timestamp	Risk	Previous Hash (H_{i-1})	Calculated Record Hash (H_i)
Genesis	-	-	-	-	-	-	GENESIS
Record 1	EVT-001	2	30	00:01.20	HIGH	GENESIS	c160fbb209f7e6c1a8b21054ff48f12c1e32d2f502c380a0bf4f9499e2787733

Record 2	EVT-002	7	108	00:04.33	HIGH	c160fbb209f...	97374a5929182ec6dc6dba05690ca66044e6fc007af3c11a8fcfee2298219af9
Record 3	EVT-003	36	177	00:07.10	HIGH	97374a59291...	441e0fcff3badcfe7705ce8004b5f0189642d3033ccc370ded52b8e0cb9dcb05

Root Hash File

- File: `output/audit_root_hash.txt`
- Value: `441e0fcff3badcfe7705ce8004b5f0189642d3033ccc370ded52b8e0cb9dcb05`
- Purpose: Represents the cryptographic commitment of the final record `$H_3$`.

3. Verification Logic & Tamper Detection

The function `verify_secure_audit_log()` in `src/app.py` (Lines 387–430) executes automatically on the Audit Log dashboard page:

- Reads `output/secure_audit_log.csv` and `output/audit_root_hash.txt`.
- Sets `previous_hash = "GENESIS"`.
- Iterates through each record, serializes the fields into JSON, prepends `previous_hash`, and recomputes SHA-256.
- Checks two conditions per row:
 - **Condition A:** Does `row["previous_hash"] == previous_hash`? If false $\rightarrow$ "Chain broken at EVT-XXX".
 - **Condition B:** Does `row["record_hash"] == calculated_hash`? If false $\rightarrow$ "Record modified: EVT-XXX".
- Final Check: Does `final $H_N == \text{expected\_root\_hash}`? If true $\rightarrow$ "✓ Audit Log Integrity VERIFIED".

What happens if an attacker modifies Record 2?

If an attacker alters Record 2 (e.g. changes `person_track_id` from `7` to `9` to cover up an intruder):

- The recomputed hash of Record 2 changes from `97374a...` to a completely different hash `a8f3d1...`.
- Record 3's stored `previous_hash` (`97374a...`) no longer matches the new Record 2 hash (`a8f3d1...`), breaking the chain.
- The final computed root hash fails to match `audit_root_hash.txt`.

- The dashboard immediately displays a red **CRITICAL TAMPER WARNING**.
-

4. SIH Judge Explanations

- **Beginner Explanation:** "Think of a hash chain like a stack of sealed transparent envelopes. Each envelope has a wax seal stamped with the code of the envelope below it. If someone opens Envelope #2 to change the paper inside, the wax seal breaks and envelope #3's code won't match anymore!"
- **Engineering Explanation:** "We construct a Merkle-like sequential cryptographic linear hash chain using SHA-256 digests over sorted JSON canonical payloads. Hash verification is $O(N)$ computational complexity."
- **SIH Judge Pitch:** "In traditional surveillance, security logs can be deleted by corrupt insiders. IBVAP guarantees zero tampered records using SHA-256 cryptographic chaining, giving military commanders legally admissible forensic evidence."